



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unitat 06 — Arrays

Bloc 1

1r CFGS DAW · Programació

Sergi García Barea



En aquesta sessió

La primera ferramenta "de veritat" per a gestionar quantitats: l'aparcament de dades

- 1 L'array: l'**aparcament de dades** — crear, indexar i la plaça fantasma.
- 2 Valors per defecte i `length` — **sense parèntesis!**
- 3 Recórrer arrays: el duo **for** i **for-each**.
- 4 Arrays **multidimensionals**: taules, taulers i matrius.
- 5 La classe **Arrays**: la navalla suïssa.

 El Bloc 2 porta la **referència** (arrays i mètodes), objectes, els reptes *Be the Code* i la cacera d'errors.

El problema: tens 100 gats i un sol nom

SenseArrays.java

```
String gato1 = "Bigotes";  
String gato2 = "Garfield";  
String gato3 = "Misifú";  
// ... 97 línies després ...  
String gato100 = "Calcetines";
```

Arriba el **gat 101** i el programa es cau. O pitjor: vols saber quants gats comencen per «M» i has d'escriure **100 if**.

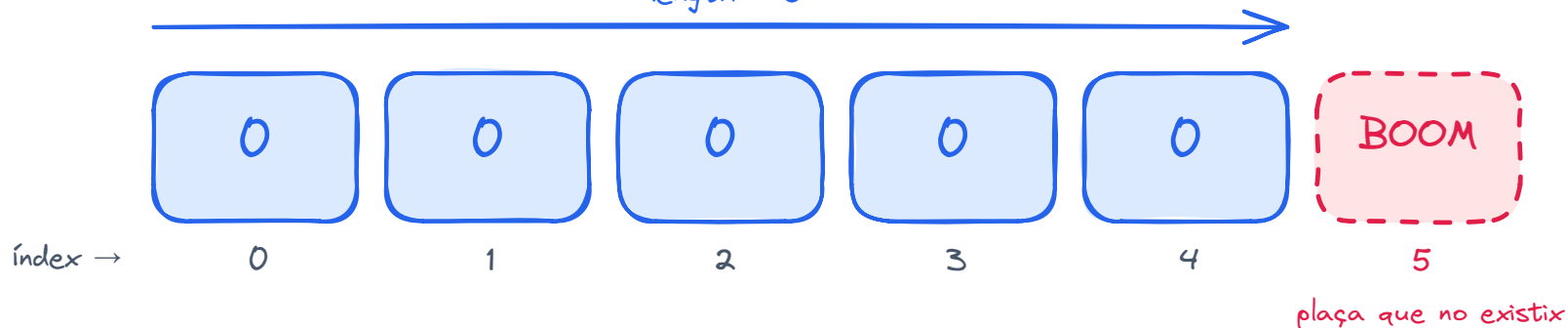


Si alguna volta escrius `gato1, gato2...`
`gatoN` al teu codi, en algun lloc un
programador sènior plora. Els arrays
existixen exactament per a això.

L'array: el teu primer aparcament

```
int[] numeros = new int[5];
```

5 places · totes comencen amb el valor per defecte (0)
length = 5



💡 l'últim element sempre és `numeros[numeros.length - 1]`

i Grandària **fixa**, un sol nom per a moltes dades del mateix tipus — i cada plaça, numerada des de 0.

Dos constructors: amb new o amb literal

●●● Buit, per a omplir després

```
// 5 places, totes buides (0)
int[] numeros = new int[5];
```

La grandària va entre claudàtors i **ja no canvia mai**.

●●● Ple, de naixement

```
// 3 places, ja ocupades
int[] directo = {10, 20, 30};
```

Els valors, entre claus i separats per comes.



La primera plaça és la **0**, no la 1. Pensa en els índexs com a **distàncies**: la primera casa és a 0 passes de tu, no a 1.

Els valors per defecte

En crear un array amb `new`, cada plaça s'ompli amb el valor per defecte del tipus:

Tipus	Valor per defecte
<code>int</code> , <code>long</code> , <code>short</code> , <code>byte</code>	<code>0</code>
<code>double</code> , <code>float</code>	<code>0.0</code>
<code>boolean</code>	<code>false</code>
<code>char</code>	<code>'\u0000'</code>
Objectes (<code>String</code> , <code>Alumne</code> ...)	<code>null</code>



Eixe últim és el que mossega: un array de `String` acabat de crear està ple de **null**, no de `""`. Cridar un mètode sobre una plaça null → `NullPointerException` a l'acte.

Ficar coses: l'índex i la plaça fantasma

S'accedix a una plaça amb claudàtors i l'**índex**. Però l'array té places del **0 al 2**: demanar la 3 és intentar aparcar on no hi ha plaça.

Parking.java

```
String[] gatos = new String[3];  
gatos[0] = "Bigotes";  
gatos[1] = "Garfield";  
gatos[2] = "Misifú";  
gatos[3] = "Calcetines"; // ¡BOOM!
```



Java respon amb

`ArrayIndexOutOfBoundsException` — la
excepció més típica d'esta unitat.



Memoritza: índexs vàlids de **0 a**
length - 1. L'últim element sempre és
`arr[arr.length - 1]`.

La longitud: length sense parèntesis

Longitud.java

```
int[] numeros = new int[10];  
System.out.println(numeros.length);  
// 10 - sense parèntesis!
```

`length` no és un mètode: és un **atribut**.

Per això no porta parèntesis.



Trampa mortal d'examen: arrays →

`length` · String → `length()` ·

col·leccions → `size()` (U12).



L'array és un **objecte** (viu al heap), però la variable que el referencia és a la pila. Quan passes un array a un mètode, passes la **referència**: si el modifiques dins, el canvi afecta l'original. Ho veurem al Bloc 2.

Comprovació ràpida

? Quant places té `int[] a = new int[7]` i quins són els índexs vàlids?

? Quin valor té cada plaça de `boolean[] b = new boolean[3]` ?

? Quina excepció llança `arr[arr.length]` ?

1. 7 places: índexs del 0 al 6 (`length - 1`). · **2.** `false` a totes: és el valor per defecte de `boolean`. · **3.** `ArrayIndexOutOfBoundsException` : la plaça `length` no existix, les vàlides acaben en `length - 1`.

Recórrer arrays: el duo inseparable

● ● ● RecorreGatos.java

```
String[] gatos = {"Bigotes", "Garfield",  
                  "Misifú", "Calcetines"};  
  
// recorrega amb index  
for (int i = 0; i < gatos.length; i++) {  
    System.out.println("Gato " + i  
        + ": " + gatos[i]);  
}
```

Gato 0: Bigotes
Gato 1: Garfield
Gato 2: Misifú
Gato 3: Calcetines

⚠ Fixa't: la condició és `i < gatos.length`. Si escrigueres `≤`, a l'última volta demanes la plaça `length` i...
¡BOOM! L'error de bucle més comés de l'univers.

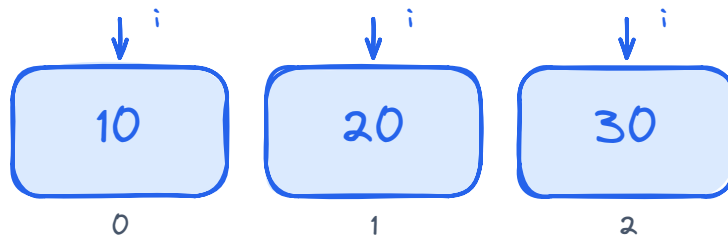
El duo, dibuixat

```
for (int i = 0; i < numeros.length; i++)
```

for clàssic

numeros

índex



lleg i modifica
`numeros[i] *= 10`

for-each

```
for (int n : numeros)
```

només LLEGeixes: *n* és una còpia — no pot modificar l'array



n pren cada valor



for clàssic: recorre per índex i pot **modificar** · **for-each:** recorre els valors i **només llig**.

Patrons clàssics amb for

Sumar tots els elements

Mitjana.java

```
int[] notas = {7, 8, 5, 9, 6};
int suma = 0;
for (int i = 0; i < notas.length; i++) {
    suma += notas[i];
}
// mitjana: (double) suma / notas.length
```

Cerca lineal

Cerca.java

```
int[] edades = {12, 45, 7, 34, 89};
int buscado = 34;
int posicion = -1;
for (int i = 0; i < edades.length; i++) {
    if (edades[i] == buscado) {
        posicion = i;
        break;
    }
}
```



Modificar l'array al lloc (`numeros[i] *= 10`) **només es pot amb índex**: el for-each és de només lectura.

for-each: la variant peresosa

ForEach.java

```
String[] gatos = {"Bigotes",  
                  "Garfield", "Misifú"};  
// recorrega sense index  
for (String gato : gatos) {  
    System.out.println("Miau: " + gato);  
}
```

Es llig: «*per a cada String gato en gatos, fes això*». Sense comptadors ni claudàtors.



El for-each és **només de lectura**: la variable del bucle és una **còpia** del valor de la plaça. `gato = "Nuevo"` només canvia la variable local, mai l'array.



Don Tip: el for-each és un robot que va per l'aparcament llegint matrícules. Només llig: no pot repintar els cotxes.

Quan use cada un?

Situació	Bucle recomanat
Només llegir i no t'importa la posició	for-each
Necessite l'índex (posicions, comparar veïns)	for clàssic
Vull modificar els elements de l'array	for clàssic
Recórrer cap arrere o de dos en dos	for clàssic
Recórrer una col·lecció (<code>ArrayList</code> , <code>HashSet</code> ...)	for-each (U12)



Si no necessites l'índex, usa **for-each**: és més curt, més llegible i t'estalvia una classe sencera d'errors (oblidar el `++` , començar en 1, escriure `≤` ...).



Be the Code: la suma dels pacients

BeTheForEach.java

```
public class BeTheForEach {  
    public static void main(String[] args) {  
        int[] numeros = {10, 20, 30, 40, 50};  
        int total = 0;  
        for (int n : numeros) {  
            if (n % 20 == 0) {  
                total += n;  
            }  
        }  
        System.out.println(total);  
    }  
}
```

Què imprimix?

a) 60 · b) 90 · c) 120 · d) 150

L'a. El for-each recorre 10, 20, 30, 40, 50 i el `if` només suma els múltiples de 20:
20 + 40 = 60. Els altres s'ignoren.

Comprovació ràpida

? Què imprimix `for (int i = 0; i < a.length; i++)` sobre `{1, 2, 3}` imprimint `a[i]` ?

? Per què `i ≤ a.length` llança excepció?

? Com imprimixes un array al revés?

1. `1 2 3` : recorre les places 0, 1 i 2. · **2.** A l'última volta (`i == length`) demana una plaça que no existix: els índexs vàlids acaben en `length - 1`. · **3.** Un for clàssic cap arrere: `for (int i = a.length - 1; i ≥ 0; i--)`.

Arrays multidimensionals: l'aparcament de plantes

Un array bidimensional és **un array de arrays**: cada plaça guarda... un altre aparcament. Es declara amb doble claudàtor:

●●● Aparcament.java

```
int[][] tabla = new int[3][4]; // 3 files, 4 columnes
// accés per fila i columna
tabla[0][0] = 1; // fila 0, columna 0
tabla[1][2] = 5; // fila 1, columna 2
```



Pensa en un aparcament amb **3 plantes** i **4 places per planta**: per a una plaça necessites **dos números**, la fila i la columna.

La matriu, dibuixada

```
int[][] tabla = new int[3][4]; // 3 files × 4 columnes
```

	col 0	col 1	col 2	col 3
Fila 0	0	0	0	0
Fila 1	0	0	5	0
Fila 2	0	0	0	0

`tabla[1][2]`
primera la fila

! `tabla.length` (files) i `tabla[0].length` (columnes) NO són el mateix — l'error clàssic amb matrius.

tabla.length → 3 files • tabla[0].length → 4 columnes
ambdues comencen en 0

Crear i recórrer una matriu

Creació amb literal

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

 Anomena els índexs `i` i `j` (o `fila` i `col`): NO uses `x` i `y` tret que treballes amb coordenades.

Bucles niats

```
for (int i = 0; i < matriz.length; i++) {  
    for (int j = 0; j < matriz[i].length; j++) {  
        System.out.print(matriz[i][j] + " ");  
    }  
    System.out.println();  
}
```

Eixida: 1 2 3 · 4 5 6 · 7 8 9

Arrays irregulars (jagged arrays)

Java permet files amb **grandàries diferents**: la fila 0 té 2 columnes, la 1 en té 5 i la 2 en té 3.

```
Irregular.java  
  
int[][] irregular = new int[3][];  
irregular[0] = new int[2];  
irregular[1] = new int[5];  
irregular[2] = new int[3];
```

 Per a què? Triangles, piràmides o dades que no formen un rectangle: els dies de cada mes (febrer en té menys).

 Per això els recorreguts usen `matriz[i].length` dins del bucle interior, **mai un número fix**.

Per a què serveixen de veritat

Situació	Array
Tauler de joc (escacs, busca-mines, tres en ratlla)	<code>char[][]</code> o <code>boolean[][]</code>
Notes per alumne i assignatura	<code>double[][]</code>
Mapa de píxels d'una imatge	<code>int[][]</code>
Matrius matemàtiques	<code>double[][]</code>

BuscaMinas.java

```
boolean[][] minas = new boolean[5][5];  
minas[2][3] = true; // hi ha una mina en fila 2, columna 3
```



Eixir-te de fila o columna torna a ser `ArrayIndexOutOfBoundsException` — ara amb dues coordenades.



Be the Code: la diagonal que no es veu

BeTheDiagonal.java

```
public class BeTheDiagonal {  
    public static void main(String[] args) {  
        int[][] m = {  
            {1, 2, 3},  
            {4, 5, 6},  
            {7, 8, 9}  
        };  
        int suma = 0;  
        for (int i = 0; i < m.length; i++) {  
            suma += m[i][i];  
        }  
        System.out.println(suma);  
    }  
}
```

Què imprimix?

a) 12 · b) 15 · c) 18 · d) 45

L'b. Suma $m[0][0] + m[1][1] + m[2][2] = 1 + 5 + 9 = 15$. Quan fila i columna són el mateix número, camines per la **diagonal principal**: un sol bucle, no dos.

Comprovació ràpida

? Quant files i columnes té `int[][] a = new int[3][4]` ?

? Com accedixes a l'element de la fila 2, columna 1?

? Què representa `a.length` i què representa `a[0].length` ?

1. 3 files i 4 columnes. · 2. `a[2][1]` : primer la fila, després la columna, tots dos començant en 0. · 3. `a.length` són les files; `a[0].length` , les columnes de la primera fila.

La classe Arrays: la teua navalla suïssa

 `java.util.Arrays` és una classe plena de **mètodes estàtics** per a treballar amb arrays: imprimir, ordenar, copiar, buscar i omplir sense escriure tu el bucle. Es criden amb el nom de la classe: `Arrays.xxx(array)`.

 ImprimirBonic.java

```
int[] numeros = {1, 2, 3};
System.out.println(numeros);
// [IQ6d06d69c ← adreça de memòria, inútil
System.out.println(Arrays.toString(numeros));
// [1, 2, 3] ← les dades, llegibles
```

 `numeros.toString()` tampoc no funciona: els arrays no sobreesciuen `toString()`. Sempre `Arrays.toString(numeros)` — i `Arrays.deepToString()` per a arrays 2D.

Ordenar i omplir: sort i fill

Arrays.sort — ordenar al lloc

Sort.java

```
int[] notas = {7, 3, 9, 5};  
Arrays.sort(notas);  
// [3, 5, 7, 9]
```

⚠ **Modifica l'original:** si vols conservar l'ordre inicial, copia abans amb `Arrays.copyOf`. Amb String ordena per Unicode: "Zebra" va abans que "abc".

Arrays.fill — omplir-ho tot

Fill.java

```
int[] tabla = new int[10];  
Arrays.fill(tabla, 7);  
// totes les places a 7
```

📄 Útil per a inicialitzar taulers, reiniciar marcadors o preparar un array abans d'usar-lo.

Buscar i copiar: `binarySearch` i `copyOf`

`Arrays.binarySearch` — buscar ràpid

● ● ● CercaBinaria.java

```
int[] nums = {3, 5, 7, 9, 11};  
int pos = Arrays.binarySearch(nums, 7);  
// 2 - partix l'array per la meitat
```

⚠ **Exigix array ordenat:** si no, el resultat és impredecible. Ordena abans de buscar, sempre.

`Arrays.copyOf` — copiar amb talla nova

● ● ● Copiar.java

```
int[] original = {1, 2, 3, 4, 5};  
int[] retallat =  
    Arrays.copyOf(original, 3);  
// {1, 2, 3}  
int[] allargat =  
    Arrays.copyOf(original, 8);  
// {1, 2, 3, 4, 5, 0, 0, 0}
```

i La forma civilitzada de «canviar la grandària» (que és fixa): en crees un de nou i copies.

Comparar: == vs Arrays.equals

Comparar.java

```
int[] a = {1, 2, 3};  
int[] b = {1, 2, 3};  
// ¿mateix objecte o mateix contingut?  
System.out.println(a == b);  
// false: són el MATEIX objecte?  
System.out.println(  
    Arrays.equals(a, b));  
// true: mateix contingut
```



```
a == b i  
a.equals(b)
```

Comparen
referències: són
el mateix objecte
en memòria?



```
Arrays.equals(a,  
b)
```

Compara el
contingut
element a
element. El teu
cap t'ho agrairà.

El mapa de la navalla

Mètode	Què fa	Compte amb
<code>Arrays.toString(arr)</code>	Imprimeix l'array llegible	No usar <code>arr.toString()</code>
<code>Arrays.sort(arr)</code>	Ordena al lloc	Modifica l'original
<code>Arrays.binarySearch(arr, v)</code>	Busca per índex	Requerix array ordenat
<code>Arrays.copyOf(arr, n)</code>	Nou array amb n elements	Crea còpia, no toca l'original
<code>Arrays.fill(arr, v)</code>	Ompli tot amb v	Servix per a inicialitzar
<code>Arrays.equals(a, b)</code>	Compara contingut	No confondre amb <code>=</code>
<code>Arrays.deepToString(arr2d)</code>	Imprimeix arrays 2D	Versió profunda del toString



Don Tip:

`binarySearch()` sense ordre és com buscar en un diccionari que no està alfabètic: no trobaràs res fiable.



Be the Code: la cerca que ho tenia tot

BeTheSort.java

```
public class BeTheSort {  
    public static void main(String[] args) {  
        int[] datos = {42, 17, 8, 99, 3};  
        Arrays.sort(datos);  
        int indice =  
            Arrays.binarySearch(datos, 42);  
        System.out.println(indice);  
    }  
}
```

Què imprimix?

a) 0 · b) 3 · c) 4 · d) 99

L'b. Després d'ordenar, l'array és {3, 8, 17, 42, 99} i el 42 està a l'índex **3**.
Sense ordenar abans, `binarySearch` podria tornar qualsevol cosa, inclòs un negatiu fals.

Comprovació ràpida

? Què torna `Arrays.binarySearch` si el valor no està a l'array?

? Què fa `Arrays.copyOf(arr, 10)` si `arr` té 4 elements?

? Per què `System.out.println(arr)` no imprimix les dades?

1. Un número negatiu: «no hi és, però ací aniria». · **2.** Crea un nou array de 10 places amb els 4 valors i la resta a 0: el truc per a «engrandir» un array. · **3.** Imprimix l'adreça de memòria (`[I@...`): per a vore les dades, `Arrays.toString(arr)`.

Resum del bloc 1 en 5 frases

- 1 Un **array** és un aparcament de **grandària fixa**: moltes dades del mateix tipus sota un nom, amb índexs des de **0**.
- 2 Índexs vàlids de **0 a length - 1**: passar-te provoca `ArrayIndexOutOfBoundsException`. I `length`, sense parèntesis.
- 3 **for clàssic**: llegir i modificar · **for-each**: només lectura. La condició segura: `i < length`.
- 4 Un array 2D és un **array de arrays**: `[fila][columna]`, bucles niats i `matriz[i].length` per a files irregulars.
- 5 La classe **Arrays**: `toString`, `sort`, `binarySearch` (ordenat!), `copyOf`, `fill` i `equals`.

